

Informe de Consolidación de Equipo

Panel de Control de Clientes (React 19 & Vite 8) —
ProyectoLyEP2026

Equipo Evaluador: Grupo 5

Integrantes: Bruno, Gonzalo, Ignacio, Lucas, Miguel

Ámbito: Legislación y Ejercicio Profesional (APU - FI UNJu)

Documento: ANALISIS.pdf

Fecha: Septiembre 2026

1 Matriz de Coincidencias y Análisis Conjunto

El presente análisis sintetiza las observaciones individuales de los cinco analistas técnicos del equipo tras la auditoría exhaustiva del repositorio base. Se identificó una alta convergencia en los defectos estructurales más graves del prototipo (seguridad en cliente, ruptura de servicios y reactividad), complementada con valiosos hallazgos singulares que fortalecen la visión holística de calidad, UX y arquitectura.

1.1 Puntos de Coincidencia (Detectados por 2 o más integrantes)

1. Exposición de credenciales y contraseñas en texto plano

Bruno

Ignacio

Lucas

Almacenamiento estático del arreglo de usuarios en `autorizacionesServices.js` empaquetado en el bundle cliente (CWE-798) y visualización abierta de `cliente.password` en `DetalleCliente.jsx` (CWE-312). Coincidencia unánime en que representa una vulnerabilidad crítica de seguridad.

2. Vulnerabilidad en control de acceso (RBAC) y doble fuente de verdad

Bruno

Ignacio

Lucas

En `Login.jsx` se guarda `role` de forma suelta en `localStorage`, `DetalleCliente.jsx` lo lee directamente evadiendo el Contexto (OWASP A01:2021) y `cerrarSesion` no lo elimina. Permite escalado de privilegios local y desincronización de estado.

3. Fragmentación y ruptura de la capa de servicios HTTP

Bruno

Ignacio

Lucas

Miguel

`clientesService.js` utiliza `Axios` únicamente para crear cliente, mientras `ListaClientes` y `DetalleCliente` realizan llamadas directas mediante `fetch` con URLs quemadas. Miguel además resalta que la aplicación no maneja reintentos ante caídas de la API.

4. Desincronización reactiva entre creación de cliente y tabla de listado

Bruno

Gonzalo

Ignacio

Lucas

`FormCliente` crea un nuevo cliente con éxito en la API mock, pero `ListaClientes` no cuenta con ningún mecanismo de actualización (callback o contexto compartido), impidiendo que el nuevo registro se refleje en pantalla sin recargar.

5. Métricas estáticas hardcodeadas y lógica muerta en Dashboard

Bruno

Ignacio

Lucas

Las tarjetas estadísticas ("Clientes: 10", "Gerencia: 3", "Soporte: 3") son valores fijos en el JSX. Asimismo, la condición `!admin ? <Login /> : ...` es código muerto inalcanzable dado que la ruta está envuelta en `RutaProtegida`.

6. Feedback bloqueante e invasivo mediante `alert()` nativo

Bruno

Gonzalo

Ignacio

Lucas

El uso de `alert('Verifique los datos')` en `Login.jsx` interrumpe el hilo de ejecución del navegador en lugar de renderizar componentes de alerta no bloqueantes de `React-Bootstrap`.

7. Filtrado vulnerable a caídas por propiedades nulas (`TypeError`)

Gonzalo

Ignacio

Lucas

El método `.filter()` en `ListaClientes.jsx` asume la existencia estricta de `cliente.name.lastname` y `cliente.address.city`, provocando caídas fatales si un dato viene indefinido. Coinciden en aplicar optional chaining.

8. Header y Nav incondicionales / Exposición de layout interno en Login

Gonzalo

Ignacio

Lucas

`App.jsx` monta la cabecera y barra de navegación de forma global, haciendo clickeable el menú privado desde `/login` y generando rebotes bruscos de redirección.

9. Monotonía visual, falta de jerarquía de color y saturación

Gonzalo

Miguel

Uso desmedido de un único tono magenta sin jerarquía de acciones (primaria, secundaria, eliminación) y ausencia de variables CSS estructuradas.

1.2 Divergencias o Hallazgos Únicos Relevantes

Discrepancia arquitectónica SPA vs Fullstack y propuesta de Backend Express

Lucas

Diagnostica que el repositorio es una SPA pura de solo lectura contra FakeStoreAPI sin persistencia real. Plantea como evolución la creación de un servidor Node.js/Express con JWT, hashing con bcrypt y persistencia propia.

Redundancia del campo "Sector" en Login & Separación de ruta /clientes/nuevo

Gonzalo

1) Detecta que exigir seleccionar "Sector" en el login es redundante y confuso, pues el sector ya pertenece al usuario registrado. 2) Señala la sobrecarga de FormCliente inline dentro de ListaClientes, proponiendo separar en /clientes y /clientes/nuevo. 3) Propone añadir íconos (react-icons) en botones y extender el buscador a nombre, email y teléfono.

Errores de ESLint, Crash por JSON.parse, CSS Pollution y falta de AbortController

Ignacio

1) Detección de 4 errores bloqueantes de ESLint (Fast Refresh y variables huérfanas). 2) Riesgo de caída fatal por JSON.parse sin try/catch en AutorizacionesContext. 3) Contaminación global de estilos (selectores form, table) y abuso de !important. 4) Falta de AbortController en useEffect. 5) Eliminación de clientes sin modal de confirmación y página 404 vacía.

Ausencia de botón "Reintentar" ante fallos de API y desalineación espacial de UI

Miguel

1) Detecta que si la API falla la pantalla queda muerta con texto estático sin botón de reintento. 2) Señala que la caja del buscador está centrada y aislada de la tabla, debiendo integrarse visualmente a la grilla. 3) Propone modo oscuro y colores semánticos estrictos para botones según su acción.

2 Tabla de Hallazgos Consolidada

ID	PROBLEMA / DEFECTO DETECTADO	ARCHIVO(S) AFECTADO(S)	DIMENSIÓN	IMPACTO	PROPUESTA DE SOLUCIÓN
HLG-01	Exposición de credenciales y contraseñas en texto claro: Usuarios mockeados en bundle cliente (CWE-798) y password expuesto en ficha de cliente (CWE-312).	autorizacionesService.js DetalleCliente.jsx	Seguridad	ALTO	Eliminar passwords del bundle; suprimir visualización de contraseñas en pantalla y preparar esquema para JWT seguro.
HLG-02	RBAC vulnerable por manipulación de localStorage y doble estado: Rol leído de localStorage.getItem('role') sin validar en Contexto.	Login.jsx DetalleCliente.jsx AutorizacionesContext.jsx	Seguridad / Estado	ALTO	Unificar rol en AutorizacionesContext. Eliminar clave suelta role y basar permisos en useAutorizaciones().
HLG-03	Capa de datos fragmentada y consumo HTTP directo sin Axios: Vistas hacen fetch nativo con URLs quemadas ignorando clientesService.js.	clientesService.js ListaClientes.jsx DetalleCliente.jsx	Arquitectura	ALTO	Centralizar operaciones CRUD en clientesService.js con cliente Axios base, timeout y variables de entorno .env.
HLG-04	Fallo de linter (ESLint con 4 errores bloqueantes): Variables no usadas y exportación que invalida Fast Refresh en Vite.	App.jsx, routes.jsx DetalleCliente.jsx AutorizacionesContext.jsx	Calidad	ALTO	Depurar variables huérfanas y reestructurar archivo de contexto para exportar únicamente el Provider.
HLG-05	Riesgo de crash por JSON.parse sin bloque try/catch: Corrupción de datos en localStorage arroja SyntaxError fatal al inicializar la app.	AutorizacionesContext.jsx	React / Estado	ALTO	Envolver inicialización en try...catch defensivo con remoción automática del ítem corrupto.
HLG-06	Aislamiento y falta de reactividad entre alta y grilla: FormCliente crea un cliente pero ListaClientes no es notificado.	FormCliente.jsx ListaClientes.jsx	React / Estado	MEDIO	Implementar callback onClienteCreado o contexto global para actualizar el estado de clientes en memoria.

ID	PROBLEMA / DEFECTO DETECTADO	ARCHIVO(S) AFECTADO(S)	DIMENSIÓN	IMPACTO	PROPUESTA DE SOLUCIÓN
HLG-07	Efectos asíncronos sin cancelación ('AbortController') ni captura: Peticiones pendientes no se abortan al desmontar y falta .catch en detalle.	ListaClientes.jsx DetalleCliente.jsx	React	MEDIO	Incorporar AbortController en limpieza de useEffect y capturar errores de red en todas las llamadas.
HLG-08	Filtrado reactivo vulnerable a caídas por datos nulos: Lectura sin validación de cliente.name.lastname arroja TypeError.	ListaClientes.jsx	React / Calidad	MEDIO	Aplicar optional chaining (cliente?.name?.lastname ?? '') y extender la búsqueda a email, nombre y teléfono.
HLG-09	Contaminación de CSS global y colisión de estilos con Bootstrap: Selectores generales sobre form, table y uso abusivo de !important.	login.css, formcliente.css listaclientes.css	Calidad / UX	MEDIO	Migrar a CSS Modules (*.module.css) o clases BEM aisladas; eliminar !important.
HLG-10	Layout rígido con cabecera y navegación expuestas en Login: Menú administrativo clickeable antes de iniciar sesión genera rebotes de ruta.	App.jsx Nav.jsx	Arquitectura / UX	MEDIO	Implementar Layout Routes con <Outlet /> en React Router para aislar el layout privado del login.
HLG-11	Métricas estáticas cableadas y código muerto en Dashboard: Tarjetas con valores fijos y rama !admin inalcanzable por RutaProtegida.	Dashboard.jsx	Arquitectura	MEDIO	Calcular métricas en base a clientes reales y eliminar bloque redundante de login.
HLG-12	Eliminación destructiva de clientes sin confirmación: Petición DELETE disparada directamente al hacer clic en el botón.	DetalleCliente.jsx	UX	MEDIO	Integrar Modal de confirmación interactivo antes de disparar la eliminación.
HLG-13	Feedback bloqueante mediante alert() nativo: Ventana emergente arcaica en error de inicio de sesión.	Login.jsx	UX	BAJO	Sustituir por componente inline <Alert variant="danger"> de React-Bootstrap.
HLG-14	Redundancia del campo "Sector" en Login: El usuario debe adivinar su propio sector cuando ya está fijado en su cuenta registrada.	Login.jsx autorizacionesServices.js	UX / Arquitectura	BAJO	Autenticar solo con email y password; asignar sector automáticamente del usuario devuelto.
HLG-15	Sobrecarga visual por FormCliente inline en ListaClientes: Se mezclan responsabilidades de creación y consulta en una sola vista.	ListaClientes.jsx	Arquitectura / UX	MEDIO	Desacoplar creación a una ruta propia /clientes/nuevo o mostrar formulario en Modal bajo demanda.
HLG-16	Página 404 desprovista de interacción y enlaces de retorno: Vista plana sin orientación ni navegación hacia el Dashboard.	ErrorPage.jsx	UX	BAJO	Diseñar página 404 asistida con botón de redirección rápida a la página principal.
HLG-17	Ausencia de reintento ante fallos de red en API y desconexión UI: Pantalla bloqueada sin botón "Reintentar"; buscador aislado de la grilla.	ListaClientes.jsx listaclientes.css	UX / UI	BAJO	Añadir botón de reintento de fetch y reposicionar el buscador alineado con la tabla.
HLG-18	Falta de iconografía en acciones y botones sin jerarquía semántica: Interfaz plana basada solo en texto sin acentos cromáticos por acción.	src/components/*.jsx src/css/*.css	UX / Calidad	BAJO	Instalar react-icons, definir paleta con variables CSS y asignar colores fijos según la criticidad de la acción.

3 Lista de Mejoras Individuales Propuestas (Opciones de Trabajo)

A continuación se delimitan cinco paquetes de trabajo independientes, asignables de manera individual a los 5 integrantes del equipo. Cada paquete especifica el alcance exacto, nivel de aislamiento, dependencias y un plan de 5 commits semánticos para su desarrollo ordenado mediante Pull Requests.

Opción 1: Centralización de Capa de Servicios API, Variables de Entorno y Manejo de Errores con Reintento

Asignación Sugerida:
Integrante 1

Código afectado: `src/services/api.js`, `src/services/clientesService.js`, `src/pages/ListaClientes.jsx`, `src/pages/DetalleCliente.jsx`, `.env`.

ALCANCE EXACTO (MODIFICAR / CREAR)

- Crear `.env` y `.env.example` (con `VITE_API_BASE_URL`).
- Crear `src/services/api.js` con instancia de Axios, timeout y baseURL.
- Extender `clientesService.js` con `getClientes`, `getClienteById`, `crearCliente`, `eliminarCliente`.
- Reemplazar `fetch` en `ListaClientes` y `DetalleCliente` por el servicio.
- Añadir botón "Reintentar" y captura de errores en vistas.

QUÉ NO SE DEBE TOCAR

- No modificar estilos CSS globales ni layout.
- No alterar la lógica de autenticación en `AutorizacionesContext`.
- No modificar el maquetado interno de `FormCliente` ni `Login`.

Independencia y Dependencias: 100% Independiente. Puede desarrollarse y testearse en su propia rama sin requerir cambios previos. Provee la base de datos limpia para las demás vistas.

PLAN DE IMPLEMENTACIÓN PASO A PASO (5 COMMITS SEMÁNTICOS)

1. **feat(env):** configurar variables de entorno `.env` y `.env.example` para URL base de API
2. **feat(api):** crear cliente centralizado axios con interceptores y timeout en `services/api.js`
3. **refactor(services):** implementar operaciones CRUD completas en `clientesService.js`
4. **refactor(views):** migrar `fetch` directo a `clientesService` en `ListaClientes` y `DetalleCliente`
5. **feat(ux):** incorporar boton de reintento de conexion y manejo de errores visible en vistas

Opción 2: Seguridad, RBAC, Saneamiento del Estado de Autenticación y Corrección de ESLint

Asignación Sugerida:
Integrante 2

Código afectado: `src/context/AutorizacionesContext.jsx`, `src/hooks/useAutorizaciones.js`, `src/pages/Login.jsx`, `src/pages/DetalleCliente.jsx`, `src/pages/Dashboard.jsx`.

ALCANCE EXACTO (MODIFICAR / CREAR)

- Blindar `JSON.parse` con `try...catch` en `AutorizacionesContext`.
- Eliminar escritura/lectura aislada de `role` en `localStorage`; unificar en Contexto.
- Suprimir campo selector "Sector" en `Login`; asignar sector automáticamente.
- Ocultar visualización de contraseñas de clientes en `DetalleCliente.jsx`.
- Reemplazar `alert()` por `<Alert>` de React-Bootstrap en `Login`.
- Purgar código muerto de `login` en `Dashboarboard.jsx` y resolver errores de ESLint.

QUÉ NO SE DEBE TOCAR

- No alterar la lógica de consumo HTTP de `clientesService`.
- No modificar la tabla de clientes ni los filtros de búsqueda.
- No alterar las hojas de estilo de `listaclientes.css` ni `nav.css`.

Independencia y Dependencias: 100% Independiente. Opera exclusivamente sobre la capa de seguridad, autenticación y manejo de sesión.

PLAN DE IMPLEMENTACIÓN PASO A PASO (5 COMMITS SEMÁNTICOS)

1. **fix(context):** proteger lectura de `localStorage` con try-catch y resolver advertencia Fast Refresh
2. **refactor(auth):** unificar permisos en `AutorizacionesContext` erradicando clave suelta `role` en storage
3. **feat(login):** simplificar formulario login sin selector redundante y reemplazar `alert` por `Alert` bootstrap
4. **security(views):** suprimir impresion de contraseñas de clientes en `DetalleCliente`
5. **refactor(dashboard):** eliminar codigo muerto condicional de login y corregir variables huérfanas de eslint

Opción 3: Arquitectura de Layouts con React Router 7, Desacople de Alta (/clientes/nuevo) y Página 404

Asignación Sugerida:
Integrante 3

Código afectado: src/App.jsx, src/routes/routes.jsx, src/components/MainLayout.jsx, src/components/Nav.jsx, src/pages/NuevoCliente.jsx, src/pages/ErrorPage.jsx.

ALCANCE EXACTO (MODIFICAR / CREAR)

- Crear src/layouts/MainLayout.jsx con Header, Nav, <Outlet /> y Footer.
- En routes.jsx, anidar rutas protegidas bajo MainLayout; dejar /login limpio.
- Separar FormCliente de ListaClientes creando la página /clientes/nuevo.
- Añadir enlace "Nuevo Cliente" en Nav.jsx y botón en ListaClientes.
- Rediseñar ErrorPage.jsx con diseño amigable y botón de redirección.

QUÉ NO SE DEBE TOCAR

- No modificar la lógica interna de validación ni envío de FormCliente.
- No alterar la lógica de autenticación en AutorizacionesContext.
- No intervenir en el consumo de API de clientesService.js.

Independencia y Dependencias: 100% Independiente. Reorganiza la jerarquía de rutas y componentes contenedores sin afectar el estado interno de datos.

PLAN DE IMPLEMENTACIÓN PASO A PASO (5 COMMITS SEMÁNTICOS)

1. **feat(layout):** crear MainLayout con Outlet para separar la navegación privada del login
2. **refactor(routes):** reestructurar routes.jsx con rutas anidadas protegidas y limpiar Navigate no usado
3. **feat(routes):** crear ruta /clientes/nuevo desacoplando el formulario de alta del listado
4. **refactor(nav):** actualizar Nav.jsx con enlace a nuevo cliente y ocultar navegación cuando no hay sesión
5. **feat(404):** rediseñar ErrorPage con orientación visual y botón de retorno al Dashboard

Opción 4: Reactividad de Clientes, Filtrado Seguro Multicampo, Debounce y Feedback Visual

Asignación Sugerida:
Integrante 4

Código afectado: src/pages/ListaClientes.jsx, src/components/FormCliente.jsx, src/hooks/useDebounce.js, src/pages/Dashboard.jsx.

ALCANCE EXACTO (MODIFICAR / CREAR)

- Implementar callback onClienteCreado para reflejar altas inmediatamente en la grilla.
- Crear src/hooks/useDebounce.js para optimizar el input de búsqueda.
- Blindar el .filter() con optional chaining ante nulos y ampliar la búsqueda a nombre, apellido, email, ciudad y teléfono.
- Reemplazar <h2>Cargando...</h2> por <Spinner> y estado vacío en ListaClientes.
- Conectar Dashboard.jsx al conteo dinámico real de clientes.

QUÉ NO SE DEBE TOCAR

- No modificar la configuración de rutas ni layouts.
- No tocar AutorizacionesContext ni la lógica de Login.
- No alterar estilos de cabecera ni barra de navegación.

Independencia y Dependencias: Independiente. Trabaja sobre la interacción y flujo de datos de clientes en las vistas existentes.

PLAN DE IMPLEMENTACIÓN PASO A PASO (5 COMMITS SEMÁNTICOS)

1. **fix(filter):** incorporar optional chaining en filtro de clientes previniendo TypeErrors ante nulos
2. **feat(search):** ampliar buscador a nombre, email y telefono e implementar hook useDebounce
3. **feat(reactivity):** sincronizar alta de clientes reactivamente con la tabla mediante callback local
4. **ui(feedback):** sustituir encabezados de carga por Spinners y componente de tabla vacía
5. **feat(dashboard):** calcular métricas dinámicas de clientes reales en el panel de control

Opción 5: Sistema de Diseño Visual (CSS Modules / Variables), Iconografía y Modal de Confirmación

Asignación Sugerida:
Integrante 5

Código afectado: `src/css/*.css`, `src/pages/DetalleCliente.jsx`, `src/pages/ListaClientes.jsx`, `src/components/Header.jsx`, `package.json`.

ALCANCE EXACTO (MODIFICAR / CREAR)

- Instalar `react-icons` y añadir íconos semánticos en botones (salir, guardar, eliminar, ver ficha).
- Definir paleta global de variables CSS (`:root`) diferenciando acciones primarias, secundarias y de peligro.
- Aislar estilos migrando a CSS Modules (`*.module.css`) y erradicar `!important`.
- Reubicar el contenedor del buscador para que esté integrado visualmente junto a la grilla.
- Implementar Modal de confirmación en `DetalleCliente.jsx` antes de ejecutar `DELETE`.

QUÉ NO SE DEBE TOCAR

- No modificar la lógica de autenticación ni enrutamiento.
- No cambiar los nombres de las funciones en `clientesService.js`.
- No alterar la estructura de datos recibida desde la API.

Independencia y Dependencias: 100% Independiente. Se enfoca en la capa de presentación, accesibilidad, feedback visual y diseño responsivo.

PLAN DE IMPLEMENTACIÓN PASO A PASO (5 COMMITS SEMÁNTICOS)

1. `style(theme)`: definir paleta de variables CSS en `app.css` con jerarquía cromática de acciones
2. `feat(Icons)`: incorporar `react-icons` en botones de navegación, guardado, eliminación y ficha
3. `refactor(styles)`: modularizar estilos de login, listado y formulario erradicando selectores globales
4. `ui(layout)`: integrar y alinear la caja del buscador contigua a la cabecera de la tabla
5. `feat(ux)`: implementar modal interactivo de confirmación previa a la eliminación de clientes